+----------------------------------+
| EE 415 |
| PROJECT 1: THREADS |
| DESIGN DOCUMENT |
+----------------------------------+

---- GROUP ----

Seryong Jang <royjang1996@gmail.com>
Ben Do <bd1228@kaist.ac.kr>

---- PRELIMINARIES ----

>> If you have any preliminary comments on your submission, notes for the
>> TAs, or extra credit, please give them here.

>> Please cite any offline or online sources you consulted while
>> preparing your submission, other than the Pintos documentation, course
>> text, lecture notes, and course staff.
We mainly used Google Gemini and Github Copilot for consulting the project.

ALARM CLOCK

---- DATA STRUCTURES ----

>> A1: Copy here the declaration of each new or changed `struct' or
>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

added to src/threads/thread.h

```
int64_t wakeup_tick;
```

added to src/threads/timer.c

```
static struct list sleep_list;
static int64_t next_wakeup;
void
timer_init (void)
{
 pit_configure_channel (0, 2, TIMER_FREQ);
 intr_register_ext (0x20, timer_interrupt, "8254 Timer");
 list_init (&sleep_list);
 next_wakeup = INT64_MAX;
}
```

```
void timer_sleep(int64_t ticks)
{
 ASSERT (intr_get_level () == INTR_ON);

 if (ticks <= 0) {
   return;
 }

 enum intr_level old_level = intr_disable (); // disable
interrupts to prevent race conditions
 struct thread *cur = thread_current ();
 cur->wakeup_tick = timer_ticks () + ticks;
 list_insert_ordered (&sleep_list, &cur->elem,
thread_wakeup_tick_less, NULL);
 if (cur->wakeup_tick < next_wakeup) {
   next_wakeup = cur->wakeup_tick;
 }
 thread_block ();
 intr_set_level (old_level);
 }
```

Note: Interrupt should be disabled— Because this is the data structure where it shares with interrupt handler, so should be intr_disable, not lock.

```
/* Timer interrupt handler. */
static void
timer_interrupt (struct intr_frame *args UNUSED)
{
 ticks++;
 thread_tick ();
 if (ticks < next_wakeup) {
   return;
 }

 for (struct list_elem *e = list_begin (&sleep_list); e !=
list_end (&sleep_list); ) {
   struct thread *t = list_entry (e, struct thread, elem);
   if (t->wakeup_tick > ticks) {
     break;
   }
   list_pop_front (&sleep_list);
```

```
   thread_unblock (t);
   e = list_begin (&sleep_list);
 }

 if (!list_empty (&sleep_list)) {
   struct thread *next_thread = list_entry (list_begin
(&sleep_list), struct thread, elem);
   next_wakeup = next_thread->wakeup_tick;
 } else {
   next_wakeup = INT64_MAX;
 }
 }
```

Note: Preferred to use list_begin than list_front().

src/threads/Make.vars
SIMULATOR = --qemu

---- ALGORITHMS ----

>> A2: Briefly describe what happens in a call to timer_sleep(),
>> including the effects of the timer interrupt handler.
timer_sleep() does the role of letting the current thread to sleep for *ticks* amount of time and yields the cpu to the next thread until it wakes up. This function should be with interrupt disabled, otherwise the system might panic.
timer_sleep() and timer_interrupt() act like a relationship of producer and consumer- let sleep and wake up threads.
timer_interrupt() is an interrupt handler which is called when timer interrupt occurs. This function increases the system's timer ticks, updates the state of the current thread and wakes up the blocked thread (sleeping or waiting thread) if needed using thread_unblock().

>> A3: What steps are taken to minimize the amount of time spent in
>> the timer interrupt handler?
First, we used next_wakeup variable. It sorts the list in timer_sleep() and in timer_interrupt(), fetches next_wakeup variable. So others cost 0(1), only the thread which will be woken up cost 0(n) to be unblocked again.

timer_interrupt() compares ticks < next_wakeup and returns immediately in that case. timer fires 100 ticks per second and most ticks wake nobody, so in most cases it costs a single 64-bit comparison and plus the existing thread_tick().

sleep_list is sorted outside the handler. timer_sleep() does the operation list_insert_ordered() in the sleeping thread's own context. The handler just traverses the sleep_list to find the first thread whose wakeup_tick is bigger than current ticks. Its cost is

O(n) in the number of threads actually woken, not O(n) in the number of all sleepers. Recalculating next_wakeup is O(1) because the front element of the list is the new minimum.

And also for MLFQS, we used macros for fixed-points operations to reduce the time in interrupt context.

---- SYNCHRONIZATION ----

>> A4: How are race conditions avoided when multiple threads call
>> timer_sleep() simultaneously?
The point is that Pintos runs on a single CPU, so disabling interrupts is enough for mutual exclusion on Pintos.
Context switch can only be triggered by an interrupt or by thread_yield()/thread_block(). If we set interrupts off, they cannot occur in the middle of the switch.
Also, why we use interrupt disabled not Lock is because sleep_list and next_wakeup are the global states shared by every caller such as kernel thread and interrupt handler share, so we do have to use intr_disable() instead of Lock.

>> A5: How are race conditions avoided when a timer interrupt occurs
>> during a call to timer_sleep()?
The dangerous timing(critical section) is between computing wakeup_tick and calling thread_block(). If a tick was serviced there, the handler could find the thread already linked into sleep_list while its status is still THREAD_RUNNING, and call thread_unblock() on a thread that is not blocked.
In timer_sleep(), intr_disable() stops the interrupt from external until the thread completely goes to sleep. It should be done atomically. So it prevents lost wakeup and intr_set_level works safely only after the thread is woken up.
Because interrupts are disabled across the whole system, the timer interrupt stays pending in the PIC and is not delivered until intr_set_level() reenables it. By then the thread is already THREAD_BLOCKED and correctly positioned in the list.

---- RATIONALE ----

>> A6: Why did you choose this design? In what ways is it superior to
>> another design you considered?
We implemented a dedicated sleep_list sorted by absolute wakeup tick, a cached next_wakeup minimum, and thread_block()/thread_unblock() to remove the sleeper from the scheduler entirely.
For an unsorted list, to make a full scan at every tick, the handler must examine every sleeper on all 100 ticks per second, with interrupts disabled. So we preferred to move that cost to the caller, since interrupt latency affects the whole system while timer_sleep() only delays its own caller.

---- TEST----

```
cd /home/roy/ee40015/pintos/src/threads/build
```

```
rm -f tests/threads/alarm-*.result tests/threads/priority-*.result

make tests/threads/alarm-single.result \
     tests/threads/alarm-multiple.result \
     tests/threads/alarm-simultaneous.result \
     tests/threads/alarm-zero.result \
     tests/threads/alarm-negative.result 2>&1 | grep -E
"^(pass|FAIL)"
```

```
roy@roy-VM:~/ee40015/pintos/src/threads/build$ cd /home/roy/ee40015/pintos/src/threads/buil
d
roy@roy-VM:~/ee40015/pintos/src/threads/build$ rm -f tests/threads/alarm-*.result tests/thr
eads/priority-*.result
roy@roy-VM:~/ee40015/pintos/src/threads/build$ make tests/threads/alarm-single.result \
     tests/threads/alarm-multiple.result \
     tests/threads/alarm-simultaneous.result \
     tests/threads/alarm-zero.result \
     tests/threads/alarm-negative.result 2>&1 | grep -E "^(pass|FAIL)"
pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
roy@roy-VM:~/ee40015/pintos/src/threads/build$
```

PRIORITY SCHEDULING
==========

---- DATA STRUCTURES ----

>> B1: Copy here the declaration of each new or changed `struct' or
>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

added to struct thread in threads/thread.h

int base_priority;
Thread’s very first original priority value set in thread_create() or thread_set_priority().
priority is recomputed from this plus donations; this is never overwritten by a donor.

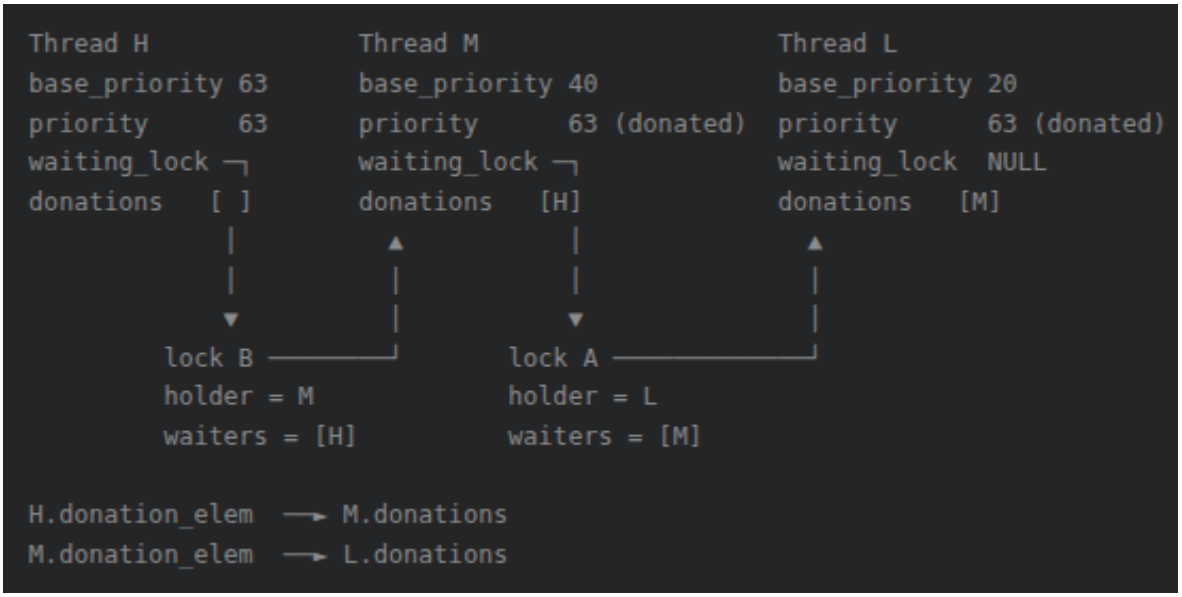
struct lock *waiting_lock;
The lock which this thread is waiting for lock_acquire().

struct list donations;
List of threads which donated priority to this thread.

struct list_elem donation_elem;
List element for the donations list which is doubly-linked to its lock holder’s donations.

No new global or static variables, typedefs, or enumerations were introduced for priority scheduling.

>> B2: Explain the data structure used to track priority donation.
>> Use ASCII art to diagram a nested donation. (Alternately, submit a
>> .png file.)



Each thread keeps the list of threads that are blocked on locks it holds. If it has several locks acquired and releases one of them, its priority must be recalculated as the biggest one among left donations. And each blocked threads record which lock they are waiting for (waiting_lock). Thread's priority is between 0~63 (min~max in pintos, -20~20 on other OS) and set as max priority in donations. Nested donation is implemented by waiting_lock->holder repeatedly. Recommended depth is 8.
When H calls lock_acquire(B), H is added to M.donations and thread_donate_priority() walks H->B.holder = M (raise to 63)->M.waiting_lock = A->A.holder = L (raise to 63) ->L.waiting_lock = NULL, stop. Both M and L now run at 63, so L can finish and release A, M then releases B, and H proceeds.

---- ALGORITHMS ----

>> B3: How do you ensure that the highest priority thread waiting for
>> a lock, semaphore, or condition variable wakes up first?
`sema_down()` inserts the caller with `list_insert_ordered()` using `thread_cmp_priority()`.
`sema_up()` additionally calls `list_sort()` on `sema->waiters` before `list_pop_front()`, because a waiter's priority may have changed while blocked (it received or lost a donation). Listing again at wake-up time ensures to find the next wakeup thread in the list without any mistake.

Locks*:* `lock_acquire()`/`lock_release()` are built on struct semaphore, actually lock is binary semaphore and sleeping lock. So they inherit the semaphore's feature and donation guarantees that the lock holder runs at the highest priority of waiters.

>> B4: Describe the sequence of events when a call to lock_acquire()
>> causes a priority donation. How is nested donation handled?
Let thread T with priority P calls `lock_acquire(L)` while L is held by holder- another thread H.

1. `lock->holder != NULL`, so donation is needed. Interrupts are disabled.
2. `T->waiting_lock = L`.
3. T is inserted into `H->donations` list via `donation_elem`, ordered by `donation_priority`.
4. `thread_donate_priority()`: starting with `cur = T`, it calls `cur->waiting_lock->holder`. If the holder's `priority` is ≥ P it stops.

   If no, it sets `holder->priority = P` and continues with `cur = holder`.

5. `sema_down()` blocks T on L's semaphore.
6. When T wakes up, and `T->waiting_lock = NULL` and sets `L->holder = T`.

`DONATION_DEPTH_LIMIT`=8

>> B5: Describe the sequence of events when lock_release() is called
>> on a lock that a higher-priority thread is waiting for.
Let thread H hold L, and let another thread T (higher priority) be blocked on L. H calls `lock_release(L)`:

1. With interrupts disabled, `lock_remove_donations(L)` walks `H->donations` and removes every donor whose `waiting_lock == L`. Donors waiting on *other* locks H still holds remain.
2. `thread_refresh_priority()` recomputes `H->priority`: reset to `base_priority`, then raise to the highest priority left in `donations`. H therefore drops back to its own priority (or to the level required by remaining donors).
3. `L->holder = NULL`.
4. `sema_up(&L->semaphore)` sorts the waiters, unblocks T (the highest), and calls `thread_preempt()`. T now is bigger than H, so H yields immediately; T resumes inside `lock_acquire()`, sets itself as holder, and runs.

---- SYNCHRONIZATION ----

>> B6: Describe a potential race in thread_set_priority() and explain
>> how your implementation avoids it. Can you use a lock to avoid
>> this race?
In our implementation, we prevented potential race by using intr_disable() in thread_set_priority() and thread_donate_priority().

`thread_set_priority()` rewrites `base_priority` and then recomputes `priority` by reading `donations`. Concurrently, a donor in `lock_acquire()` writes the same `priority` field directly from `thread_donate_priority()`.

We can’t use lock in interrupt context because threads can't sleep/block inside interrupt handler so it's not allowed to use lock_acquire().

Lock is actually binary semaphore in Pintos and also sleeping lock, if we try to use lock to protect priority_donation logic, then we might be into infinite recursion. (like egg and chickens)

```
ASSERT (!intr_context ());
```

---- RATIONALE ----

>> B7: Why did you choose this design? In what ways is it superior to
>> another design you considered?

The key decision was to store donations as a list of *donor threads* on the holder, rather than as a single donated value or a list of locks.

---- TEST ----

cd /home/roy/ee40015/pintos/src/threads && make -s
cd build
rm -f tests/threads/alarm-*.result tests/threads/priority-*.result
make $(for t in alarm-single alarm-multiple alarm-simultaneous alarm-zero \
            alarm-negative alarm-priority priority-change priority-preempt \
            priority-fifo priority-sema priority-condvar; do
        echo tests/threads/$t.result; done) 2>&1 | grep -E "^(pass|FAIL)"

```
^Croy@roy-VM:~/ee40015/pintos/src/threads/buildcd /home/roy/ee40015/pintos/src/threads && make -s-s
cd build
rm -f tests/threads/*.result
make $(for t in alarm-single alarm-multiple alarm-simultaneous alarm-zero \
   alarm-negative alarm-priority priority-change priority-preempt priority-fifo \ …
pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
pass tests/threads/alarm-priority
pass tests/threads/priority-change
pass tests/threads/priority-preempt
pass tests/threads/priority-fifo
pass tests/threads/priority-sema
pass tests/threads/priority-condvar
pass tests/threads/priority-donate-one
pass tests/threads/priority-donate-multiple
pass tests/threads/priority-donate-multiple2
pass tests/threads/priority-donate-nest
pass tests/threads/priority-donate-sema
pass tests/threads/priority-donate-lower
pass tests/threads/priority-donate-chain
roy@roy-VM:~/ee40015/pintos/src/threads/build$
```

ADVANCED SCHEDULER

==========

---- DATA STRUCTURES ----

>> C1: Copy here the declaration of each new or changed `struct' or
>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.
fixed-point.h

```
typedef int32_t fixed_t;
```

Represent fixed-point values used for load_avg and recent_cpu operations.

thread.h

```
fixed_t recent_cpu;
```

Stores recent_cpu variable (recent CPU usage for threads) for MLFQS priority calculations.

thread.h

```
int nice;
```

Nice values from 0~63
thread.c

```
static fixed_t load_avg;
```

systemwide load average (related to how many threads are in ready_list or running in the whole system)
thread.c

```
bool thread_mlfqs;
```

indicates whether mlfqs is enabled.

---- ALGORITHMS ----

>> C2: Suppose threads A, B, and C have nice values 0, 1, and 2. Each
>> has a recent_cpu value of 0. Fill in the table below showing the
>> scheduling decision and the priority and recent_cpu values for each
>> thread after each given number of timer ticks:

Priority = PRI_MAX – (recent_cpu/4) – (nice*2)
recent_cpu = (2*load_avg)/(2*load_avg+1)*recent_cpu + nice
load_avg = (59/60)*load_avg + (1/60)*ready_threads
Let load_avg = 0.

timer	recent_cpu			priority			thread
ticks	A	B	C	A	B	C	to run
-----	--	–	–	–	–	–	------
0	0	0	0	63	61	59	A
4	4	0	0	62	61	59	A
8	8	0	0	61	61	59	B
12	8	4	0	61	60	59	A
16	12	4	0	60	60	59	B
20	12	8	0	60	59	59	A

24	16	8	0	59	59	59	C
28	16	8	4	59	59	58	B
32	16	12	4	59	58	58	A
36	20	12	4	58	58	58	C

>> C3: Did any ambiguities in the scheduler specification make values
>> in the table uncertain? If so, what rule did you use to resolve
>> them? Does this match the behavior of your scheduler?
Actually, there was not any uncertainty as the schedular specification is so obvious and clear, and the schedular (mlfqs) works according to it well. To add more words, in tie-breaking situation, Priority A = Priority B, then we can use FIFO/Round-Robin so pop the first thread in ready_list.

>> C4: How is the way you divided the cost of scheduling between code
>> inside and outside interrupt context likely to affect performance?
Inside Interrupt context, we must try to perform the as least operations as possible because the longer and complicated this code is, interrupt latency gets longer and it leads to the low responsiveness of the whole system and delayed I/O handling.
But in normal thread context, it's outside of interrupt context and interrupt enabled, so even if we do any long complicated operations, it does not affect hardware interrupts from outside.
In our implementation, lightweight operations such as incrementing recent_cpu for the running thread are performed on every timer tick inside the interrupt context. However, periodic calculations such as updating recent_cpu for all threads every 100 ticks and updating priorities every 4 ticks, and also load_avg every one sec, the calculations require iterating over all_list, incurring O(N) time complexity within the interrupt context. But for Pintos, with a small number of threads, our implementation is the acceptable performance.

---- RATIONALE ----

>> C5: Briefly critique your design, pointing out advantages and
>> disadvantages in your design choices. If you were to have extra
>> time to work on this part of the project, how might you choose to
>> refine or improve your design?
Actually, the deadline was a bit tight for us. In our design, the multiple ready queue was not implemented, used macros instead of inline functions. And we might try to find better ways for a more advanced schedular than mlfqs.

>> C6: The assignment explains arithmetic for fixed-point math in
>> detail, but it leaves it open to you to implement it. Why did you
>> decide to implement it the way you did? If you created an
>> abstraction layer for fixed-point math, that is, an abstract data
>> type and/or a set of functions or macros to manipulate fixed-point
>> numbers, why did you do so? If not, why not?

```
typedef int32_t fixed_t;
```

```c
#define F (1 << 14) /* Fixed-point scaling factor */

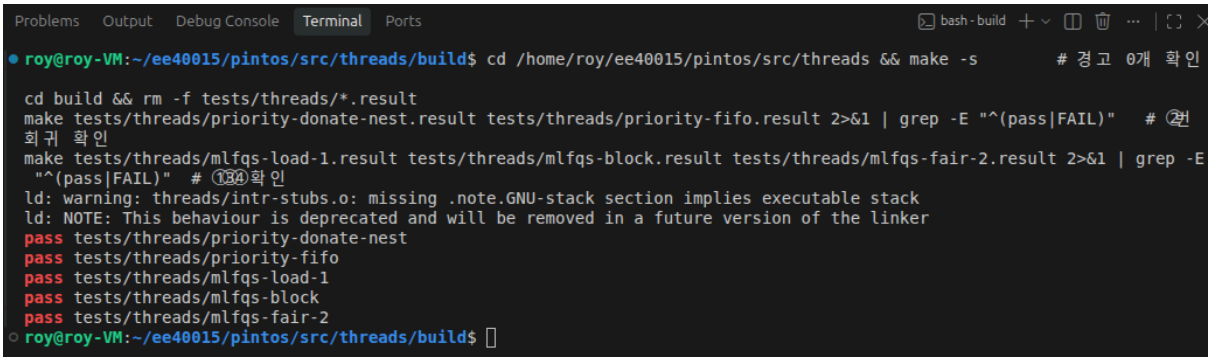
/* conversion between integers and fixed-point numbers */
#define INT_TO_FP(n) ((fixed_t)(n) * F)
#define FP_TO_INT_ZERO(x) ((x) / F)
#define FP_TO_INT_NEAREST(x) (((x) >= 0) ? (((x) + F / 2) / F) :
(((x) - F / 2) / F))

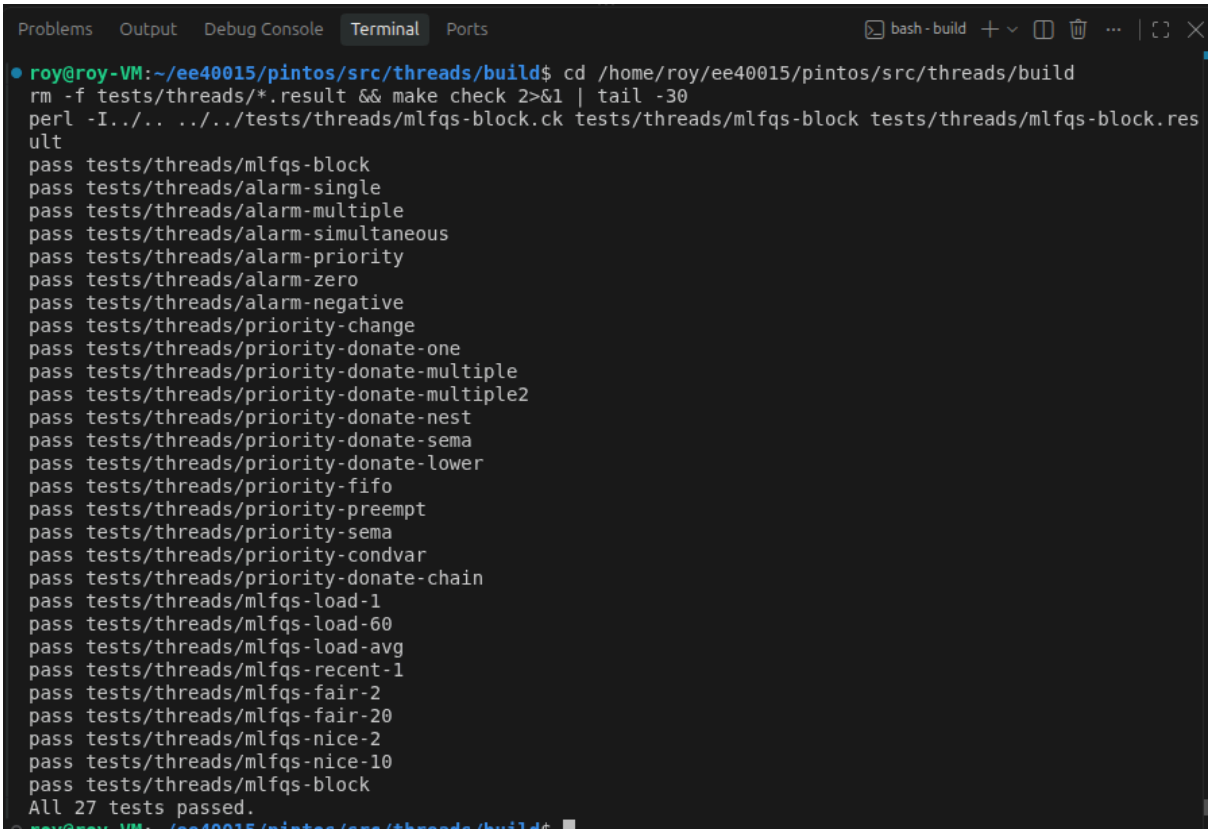
/* arithmetic operations for fixed-point numbers */
#define ADD_FP(x, y) ((x) + (y))
#define SUB_FP(x, y) ((x) - (y))
#define ADD_MIX(x, n) ((x) + (n) * F)
#define SUB_MIX(x, n) ((x) - (n) * F)
#define MUL_FP(x, y) ((fixed_t)(((int64_t)(x)) * (y) / F))
#define MUL_MIX(x, n) ((x) * (n))
#define DIV_FP(x, y) ((fixed_t)(((int64_t)(x)) * F / (y)))
#define DIV_MIX(x, n) ((x) / (n))
```

We have to prevent interger overflow by using int64 casting in priority calculation operations. Fixed points multiply operations such as x*y or x*F/y may occur overflow exceeding the scope of int32.

We also have to reduce the time in the interrupt handler as much as possible. So we decided to use macros to maximize the performance of the code (Register Allocation) as macros do inlining into caller functions such as timer_interrupt when the compiler reads source code and we can reduce the number of assembly operations significantly.

cd /home/roy/ee40015/pintos/src/threads && make -s
cd build && rm -f tests/threads/*.result
make tests/threads/priority-donate-nest.result tests/threads/priority-fifo.result 2>&1 | grep -E "^(pass|FAIL)"
make tests/threads/mlfqs-load-1.result tests/threads/mlfqs-block.result tests/threads/mlfqs-fair-2.result 2>&1 | grep -E "^(pass|FAIL)"





SURVEY QUESTIONS
==========

Answering these questions is optional, but it will help us improve the course in future quarters. Feel free to tell us anything you want--these questions are just to spur your thoughts. You may also choose to respond anonymously in the course evaluations at the end of the quarter.

>> In your opinion, was this assignment, or any one of the three problems
>> in it, too easy or too hard? Did it take too long or too little time?

>> Did you find that working on a particular part of the assignment gave
>> you greater insight into some aspect of OS design?

>> Is there some particular fact or hint we should give students in
>> future quarters to help them solve the problems? Conversely, did you
>> find any of our guidance to be misleading?

>> Do you have any suggestions for the TAs to more effectively assist
>> students, either for future quarters or the remaining projects?

>> Any other comments?